COMP2113 OPERATING SYSTEMS · WEEK 6

Semaphores and the Readers-Writers Problem

From a naive lock to a correct, efficient solution — and what breaks along the way

Today's Roadmap: One Problem, Two Bugs, One Solution

1 The Problem

What are readers and writers? Why can't we just lock everything?

2 Attempt 1: One Big Lock

Naive solution works — but is far too slow. Why?

3 Attempt 2: A Reader Counter

First reader locks out writers; last reader unlocks. But the counter itself has a race condition.

4 The Fix: A Second Semaphore

Protect the counter. This gives the reader-preference solution.

5 Trace It by Hand

Watch semaphore values change step by step.

6 The Catch: Writer Starvation

Correct and fast — but writers can wait forever. Trade-off matters.

Introducing Semaphores: A Sleepable Lock, in Two Flavors

Last class we used disabling interrupts and spinlocks. A semaphore adds one thing: a blocked thread **sleeps** instead of spinning.

1. Counting Semaphore

Value ranges from **0 to n**. It manages **n identical resource instances.**

- S = number of resources still free
- Up to n threads can hold the resource at once
- Example: a pool of 5 printers, or 3 database connections
- Used for resource allocation, not just locks

2. Binary Semaphore

Value is only **0 or 1**. Used for **mutual exclusion** — one thread in the critical section.

- 1 = lock free, 0 = lock held
- Like a mutex, but with one difference: no ownership — any thread may signal()
- Example: protecting a single critical section
- This is what we use today

*In today's reader-writer code, both **x** and **wsem** are binary semaphores initialized to 1.*

Abstract Value vs Internal Counter: Why You May See Negative Numbers

What you, the programmer, see

Counting semaphore: $S \in \{0, 1, ..., n\}$
S = number of free resources

Binary semaphore: $S \in \{0, 1\}$
1 = free, 0 = held

What the OS does internally

To avoid busy-wait, blocked threads **sleep on a queue.**

In this blocking implementation, the internal counter may drop **below 0:** |S| = number of sleeping threads.

Example: a binary mutex S = 1 (trace of the internal counter):

Action	Internal S	Meaning
A: wait(S)	0	A holds the lock; no waiters
B: wait(S) → blocks	-1	One thread (B) asleep on the queue
C: wait(S) → blocks	-2	Two threads (B, C) asleep
A: signal(S) → wake B	-1	B wakes; C still asleep

Negative numbers are not a third semaphore type — they are how the OS tracks waiting threads inside the same binary/counter semaphores.

Inside wait() and signal(): The Waiting Queue

A semaphore is not just a number. It is a number plus a queue of sleeping threads.

```
// data structure
typedef struct {
int value;
PCB *queue; // sleeping threads
} sem;

// wait(S) — P operation
S.value = S.value - 1;
if (S.value < 0) {
enqueue(S.queue, current);
block(current); // sleep
}
```

```
// signal(S) — V operation
S.value = S.value + 1;
if (S.value <= 0) {
PCB *p = dequeue(S.queue);
wakeup(p); // make p ready
}

// after waking, p resumes
// right where it blocked.
```

The internal condition — why wait checks < 0 but signal checks <= 0

wait(): we decrement first. If the result is negative, no resource was available — block. signal(): we increment first. If the result is still ≤ 0, at least one thread was already waiting — wake one up. No busy-wait: block() puts the thread to sleep and the scheduler runs another.

CHAPTER 1 · THE PROBLEM

If a file is being read by 100 students, do we really need to lock it like a bank vault?

Readers never change the data. Writers do. Treating them the same throws away most of the possible parallelism — but treating them differently opens a can of race-condition worms.

The Problem: Two Kinds of Threads, One Shared Resource

Readers

Only read the shared data. They never modify it. Multiple readers can be inside the critical section at the same time — they do not interfere.

Writers

Read and modify the shared data. A writer needs exclusive access — no other reader or writer may be active while it writes.

Allowed combinations (who may be inside the resource at the same time):

	Another Reader	Another Writer
A Reader	ALLOWED	FORBIDDEN
A Writer	FORBIDDEN	FORBIDDEN

Attempt 1: One Big Lock for Everyone

The simplest idea: one mutex semaphore, and everyone waits on it before touching the file.

```
semaphore wsem = 1;

// Reader
wait(wsem); READ FILE; signal(wsem);

// Writer
wait(wsem); WRITE FILE; signal(wsem);
```

Correct

Writer-writer and reader-writer overlap are both impossible. No race condition.

Way too slow

Only ONE reader at a time. We forbid the very reader-reader concurrency that is safe.

The Key Insight: Track How Many Readers Are Inside

Introduce a shared counter **readcount** = number of readers currently reading.

First reader arrives

readcount goes 0 → 1. This reader must acquire wsem to block any writer.

Middle readers arrive / leave

readcount is 2, 3, ... No need to touch wsem. Readers enter/leave freely.

Last reader leaves

readcount goes 1 → 0. This reader must signal wsem to release the writer lock.

Writers still always do wait(wsem) before writing and signal(wsem) after.

Attempt 2: Draft Code — Readcount Without Protection

```
int readcount = 0;
semaphore wsem = 1;

// Reader
readcount = readcount + 1;
if (readcount == 1)
wait(wsem);

READ FILE;

readcount = readcount - 1;
if (readcount == 0)
signal(wsem);

// Writer
wait(wsem); WRITE FILE; signal(wsem);
```

What is wrong here?

readcount is a **shared variable.** The operation readcount++ is NOT atomic.

It compiles to three steps: **load** → **increment** → **store.** Two readers can interleave these steps and overwrite each other's update.

Result: readcount becomes wrong. We may skip wait(wsem) or signal(wsem) — breaking correctness.

A Concrete Interleaving That Breaks the Draft

Initial state: readcount = 0, wsem = 1. Two readers R1 and R2 arrive at nearly the same time.

Step	Action	readcount	wsem
1	R1: load readcount → reg=0	0	1
2	R2: load readcount → reg=0	0	1
3	R1: reg=1, store → readcount=1	1	1
4	R2: reg=1, store → readcount=1	1 (should be 2!)	1
5	R1: sees 1==1 → wait(wsem) → wsem=0	1	0
6	R2: sees 1==1 → wait(wsem) → BLOCKED	1	0 — R2 waits forever

The lost update in step 4 makes R2 think it is the first reader. It blocks on wsem — which R1 already holds — and never starts reading.

The Fix: One More Mutex to Protect the Counter

The fix is conceptually simple: every access to readcount must itself be a critical section. Add a second mutex semaphore **x**, initialized to 1, and wrap every readcount update in wait(x) ... signal(x).

Semaphore x = 1

Protects ONLY the counter:
readcount++
readcount--
the test readcount == 1 / == 0

It is held for microseconds, released immediately. It never covers the actual READ.

Semaphore wsem = 1

Protects the shared file itself:
Acquired by the FIRST reader (blocks writers)
Released by the LAST reader
Always acquired/released by every writer

Multiple readers inside the file at once is the whole point.

Final Solution: Reader-Preference Code

```
int readcount = 0;
semaphore x = 1; // protects readcount
semaphore wsem = 1; // resource / writer lock

// ---------- Reader ----------
wait(x); // enter counter section
readcount = readcount + 1;
if (readcount == 1)
wait(wsem); // first reader locks writers out
signal(x); // leave counter section

READ FILE; // NOT under x; readers run concurrently

wait(x);
readcount = readcount - 1;
if (readcount == 0)
signal(wsem); // last reader lets writers in
signal(x);

// ---------- Writer ----------
wait(wsem);
WRITE FILE;
signal(wsem);
```

Case Study: Three Common Mistakes in Reader Code

Each variant below "looks almost right" but breaks a key property. Can you spot the consequence before reading on?

Mistake 1

Wrap READ inside x

```
wait(x); rc++;
if rc==1 wait(wsem);
READ FILE; // inside x!
signal(x);
```

Consequence:
x is held during the whole read, so only one reader reads at a time. The entire point — concurrent reads — is lost.

Mistake 2

Drop the if-check

```
wait(x); rc++;
wait(wsem); // always!
signal(x);
READ; ...
```

Consequence:
Every reader takes wsem. The second reader blocks on wsem held by the first — readers are serialized, just like the naive solution.

Mistake 3

Forget signal(x)

```
wait(x); rc++;
if rc==1 wait(wsem);
// signal(x); // missing!
```

Consequence:
First reader never releases x. The second reader blocks on wait(x) forever. Only one reader ever enters — and the system can deadlock on the exit path.

Trace 1: Two Readers Arrive First, Then a Writer

Initial: readcount = 0, x = 1, wsem = 1. Arrival order: R1, R2, W1, R1 done, R2 done.

#	Action	readcount	x	wsem	Note
1	R1: wait(x); rc=1; wait(wsem); signal(x)	1	1	0	R1 is first; locks writers out
2	R2: wait(x); rc=2; (rc!=1) skip; signal(x)	2	1	0	R2 joins; no wsem touch
3	W1: wait(wsem) —> BLOCKS	2	1	0	Writer waits; readers read concurrently
4	R1 done: wait(x); rc=1; (rc!=0) skip; signal(x)	1	1	0	Not last; keep wsem
5	R2 done: wait(x); rc=0; signal(wsem); signal(x)	0	1	1	Last reader releases wsem; W1 wakes
6	W1: writes, then signal(wsem)	0	1	1	Mission accomplished

Trace 2: A Writer Is Active; Readers and Another Writer Arrive

Initial: W1 is writing (wsem=0, rc=0, x=1). Then R1, R2, W2 arrive in that order.

#	Action	readcount	x	wsem	Note
1	R1: wait(x); rc=1; wait(wsem) BLOCKS	1	0 (held by R1)	0	R1 blocked on wsem, still holds x
2	R2: wait(x) —> BLOCKS	1	0	0	R2 queued on x
3	W2: wait(wsem) —> BLOCKS	1	0	0	W2 queued on wsem
4	W1 done: signal(wsem) —> R1 wakes	1	0	0 (held by R1)	Reader wins the wsem handoff
5	R1: signal(x); R2 enters: rc=2; signal(x)	2	1	0	Both readers read concurrently
6	W2 remains blocked on wsem	2	1	0	Writer waits until both readers leave

Limitation: Reader-Preference Can Starve Writers

What is starvation?

A thread is ready to run and holds no resources, but is perpetually denied service by the scheduler. It is not deadlocked — it is simply never chosen.

Why it happens here

While readers are active, new readers are allowed to **join** without waiting. If readers keep arriving, readcount never drops to 0, so wsem is never released.

Concrete scenario (timeline):

Time	Event	State
t1	Reader R1 starts reading (takes wsem)	rc=1, wsem=0, writer W blocked
t2	Readers R2, R3, R4 arrive and join	rc=4, W still blocked
t3	Old readers leave, but new readers keep arriving	**rc never reaches 0; W waits indefinitely**

This is a known trade-off, not an unfinished bug.

It is **not** deadlock: no circular wait exists. A second variant — **writer-preference** — blocks new readers once a writer is waiting, at the cost of reader throughput. Which variant to use is a design choice, not a correctness fix.

The Other Side: Writer-Preference Solution

To stop writer starvation, we close the gate on new readers once a writer is waiting.

Reader-Preference (what we built)

While readers are active, new readers join freely.

Writers must wait until the last reader leaves.

Risk: writer starvation.

Writer-Preference (the fix)

Once a writer is waiting, new readers are blocked.

Readers already inside finish; the waiting writer goes next.

Risk: reader starvation instead.

The big picture — there is no free lunch

Both solutions are correct. They differ only in who is favored when readers and writers compete. Which to choose depends on workload: reads-heavy workloads favor reader-preference; writes-heavy workloads favor writer-preference. The full writer-preference code uses an extra "gate" semaphore to block new readers — you will see it in the tutorial.

Key Takeaways

1. **Two semaphores, two jobs.** x (init 1) protects readcount only. wsem (init 1) protects the file / blocks writers. Do not swap their roles.

2. **Boundary-only locking.** wait(wsem) happens only when readcount goes 0→1. signal(wsem) happens only when readcount goes 1→0. Middle readers touch neither.

3. **The READ itself is outside the x-lock.** That is the whole point — x is released before reading, so many readers can read concurrently. Writers always hold wsem exclusively.

4. **Every shared variable needs protection.** The counter itself was the hidden critical section. Any read-modify-write on shared state is a race condition unless wrapped in a mutex.

5. **This is reader-preference.** It allows concurrent readers but can cause **writer starvation**. Correctness comes with a trade-off — there is no free lunch.

Practice 1: True or False?

Think first — do not peek.

Q1. In the reader-preference solution, multiple readers may read the shared file concurrently.

Q2. Semaphore x is used to block writers while readers are active.

Q3. Semaphore wsem should be initialized to 0.

Q4. The actual READ FILE operation is protected by semaphore x.

Q5. The reader-preference solution guarantees no writer is ever delayed indefinitely.

Q6. If readcount equals 3 when a reader finishes, that reader must call signal(wsem).

Practice 2: Short Analysis Questions

Q1. What goes wrong if we delete semaphore **x** entirely? Give a concrete interleaving.

Q2. Why does the **first** reader call wait(wsem), but the second reader does not? What would happen if every reader called wait(wsem)?

Q3. Define writer starvation in one sentence. Why is it possible in the reader-preference solution we studied?

Q4. Suppose readcount = 4 when a reader finishes its read. Does that reader call signal(wsem)? Explain why or why not. What if readcount was 1?

WRAP-UP

Every shared variable is a lock.
Every lock has a cost.

Next week: the Dining Philosophers problem and deadlock.

